Crew AI Programming Team

An autonomous AI engineering team that turns plain-English requirements into working software

---

Project report — architecture & worked example

Built & orchestrated by **Triaz Malik**

Stack: CrewAI • Claude (Sonnet 4.5, Haiku 4.5) • OpenAI gpt-4o • Gradio / Streamlit

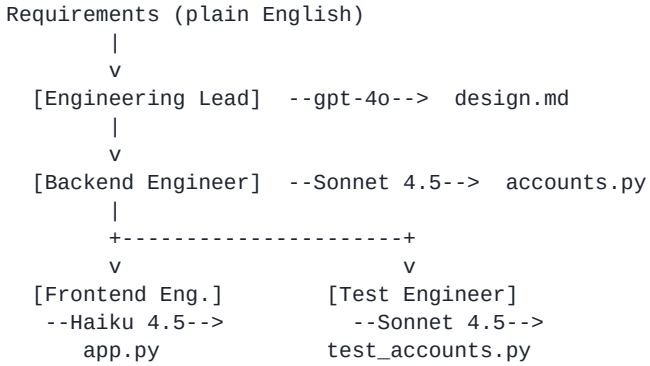
1. What is it?

The **Crew AI Programming Team** is a multi-agent system built on **CrewAI**. You describe what you want built in plain English; a small team of specialised AI agents then collaborates to deliver it — a technical design, a self-contained Python module, a Gradio demo UI, and a matching suite of unit tests. The agents run as a pipeline, each one building on the previous agent's output, mirroring how a real engineering team hands work down the line.

Everything the team produces is written to the `output/` folder, ready to run. Two front-ends are provided to drive the team: a **Gradio** app and a **Streamlit** dashboard.

2. Architecture

The system follows a **sequential** process. The Engineering Lead designs the solution; the Backend Engineer implements it; then the Frontend and Test engineers both consume the finished code as context to build a demo UI and unit tests respectively.



The team

Agent	Role	Model	Output

Engineering Lead	Turns requirements into a detailed design	`gpt-4o`	*_design.md
Backend Engineer	Implements the design in one self-contained module	`claude-sonnet-4-5`	accounts.py
Frontend Engineer	Builds a simple Gradio demo UI	`claude-haiku-4-5`	app.py
Test Engineer	Writes unit tests for the backend module	`claude-sonnet-4-5`	test_accounts.py

Cost control. Each role uses the cheapest model that does the job well: Claude **Haiku 4.5** for the boilerplate-heavy Gradio UI, Claude **Sonnet 4.5** for code and tests, and gpt-4o only for the short design step. Code-writing agents get a large output budget (max_tokens=16000) so long files are never truncated. A typical full run costs a few cents.

3. Worked example

3.1 What was requested

A trading-simulation **account management system**: create an account, deposit / withdraw funds, buy / sell shares, and report portfolio value, profit/loss, holdings, and transaction history — with guardrails that prevent overdrawing, over-buying, or selling shares the user doesn't own. A `get_share_price(symbol)` helper returns fixed prices for AAPL, TSLA and GOOGL.

3.2 Design produced by the Engineering Lead

The lead returns a markdown design listing every class and method signature — signatures only, no implementation:

```
class Account:
    def __init__(self, account_id: str, initial_deposit: float): ...
    def deposit(self, amount: float) -> None: ...
    def withdraw(self, amount: float) -> None: ...
    def buy_shares(self, symbol: str, quantity: int) -> None: ...
    def sell_shares(self, symbol: str, quantity: int) -> None: ...
    def calculate_portfolio_value(self) -> float: ...
    def calculate_profit_or_loss(self) -> float: ...
    def get_holdings(self) -> dict: ...
    def get_transactions(self) -> list: ...

def get_share_price(symbol: str) -> float: ...
```

Source: output/accounts.py_design.md

3.3 Backend code (Backend Engineer)

```
from typing import Dict, List, Tuple
from datetime import datetime


def get_share_price(symbol: str) -> float:
    """
    Retrieve the current price for a given stock symbol. This function is externally provided.

    :param symbol: The stock symbol for which to get the price
    :returns: Current price of the share
    """
    # Test implementation with fixed prices
    prices = {
```

```
            'AAPL': 150.0,
            'TSLA': 750.0,
            'GOOGL': 2800.0
        }
        return prices.get(symbol, 0.0)


    class Account:
        """
        A class used to represent a user account in a trading simulation platform.
        """
```

Source: output/accounts.py

3.4 Gradio demo UI (Frontend Engineer)

```
    import gradio as gr
    from accounts import Account

    account = None


    def create_account(account_id: str, initial_deposit: float):
        global account
        try:
            deposit_amount = float(initial_deposit)
            if deposit_amount <= 0:
                return "Error: Initial deposit must be positive"
            account = Account(account_id, deposit_amount)
            return f"Account created successfully!\nAccount ID: {account_id}\nInitial Balance: ${deposit_amo
        except ValueError as e:
            return f"Error: {str(e)}"


    def deposit_funds(amount: float):
        global account
        if account is None:
            return "Error: No account created yet"
        try:
            amount = float(amount)
```

Source: output/app.py

3.5 Unit tests (Test Engineer)

```
    import unittest
    from datetime import datetime
    from accounts import Account, get_share_price


    class TestGetSharePrice(unittest.TestCase):
        """Test cases for get_share_price function"""

        def test_get_share_price_aapl(self):
            """Test getting price for AAPL"""
            self.assertEqual(get_share_price('AAPL'), 150.0)

        def test_get_share_price_tsla(self):
            """Test getting price for TSLA"""
            self.assertEqual(get_share_price('TSLA'), 750.0)

        def test_get_share_price_googl(self):
            """Test getting price for GOOGL"""
            self.assertEqual(get_share_price('GOOGL'), 2800.0)

        def test_get_share_price_invalid_symbol(self):
            """Test getting price for invalid symbol"""
            self.assertEqual(get_share_price('INVALID'), 0.0)
```

Source: output/test_accounts.py

4. How to run

```
# 1. Install
pip install -e .

# 2. Add API keys to .env (ANTHROPIC_API_KEY + OPENAI_API_KEY)

# 3a. Drive the team from a web UI
python gradio_app.py            # Gradio
streamlit run streamlit_app.py  # or Streamlit

# 3b. …or from the CLI
crewai run

# 4. Try the generated app
cd output && python app.py
```